

Bases de Datos

Clase 4: Diseño de Bases de Datos II

Diseño de base de datos

Análisis de requisitos

Usuarios



Requisitos

Diseño conceptual de bases de datos

Requisitos



Modelo entidad-relación

Diseño lógico de bases de datos

Modelo entidad-relación



Modelo relacional

Hasta ahora

- Sabemos cómo transformar los requisitos de usuario en un modelo entidad-relación (E/R)
 - *¿Qué hacemos con este modelo?*
- **Hoy veremos cómo transformar el modelo E/R al modelo relacional**

Llaves

Modelo Relacional y Llaves

Modelo Relacional

- Los datos se almacenan como tablas:

Películas

ID Película	Nombre Película	Año	Categoría	Calificación (IMDB)
1	Interstellar	2014	Fantasía	8.6
2	The Revenant	2015	Drama	8.1
3	The Imitation Game	2014	Biografía	8.1
4	The Theory of Everything	2014	Biografía	7.7

- Distinguimos:
 - Relaciones: a cada tabla le llamamos relación
 - Atributos: son las columnas de la relación
 - Tuplas: son las filas de la relación

Modelo Relacional

Una relación es un conjunto de tuplas

- No hay filas repetidas

ID Película	Nombre Película	Año	Categoría	Calificación (IMDB)
1	Interstellar	2014	Fantasía	8.6
2	The Revenant	2015	Drama	8.1
3	The Imitation Game	2014	Biografía	8.1
4	The Theory of Everything	2014	Biografía	7.7
4	The Theory of Everything	2014	Biografía	7.7

Modelo Relacional

Una relación es un conjunto de tuplas, no hay filas repetidas

ID Película	Nombre Película	Año	Categoría	Calificación (IMDB)
1	Interstellar	2014	Fantasía	8.6
2	The Revenant	2015	Drama	8.1
3	The Imitation Game	2014	Biografía	8.1
4	The Theory of Everything	2014	Biografía	7.7
4	The Theory of Everything	2014	Biografía	7.7

× Fila duplicada — no permitido

Llaves

Ejemplo: una tabla con información de vinos

Esquema:

Vinos(Productor, Cepa, Gama, Año, Grados, Orígen)

Productor	Cepa	Gama	Año	Grados	Orígen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Tarapaca	Merlot	Reserva	2011	14	San Pedro
Viu Manent	Carmenere	Gran Reserva	2014	12	Colchagua
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	14	Maipo
...	...	...	...	...	...

Restricciones de integridad

- Son restricciones que imponemos a un esquema que todas las instancias deben satisfacer
- La restricción más importante son las llaves
- Intuitivamente, una llave permite identificar de manera única una tupla de la relación
- **Hay distintas categorías de llaves**

Llaves

Super llave

- La super llave para una relación R es:
 - *un conjunto de atributos de R tal que no pueden existir dos tuplas en R con los mismos valores de estos atributos*
- Intuitivamente: si conozco los valores de los atributos de la super llave, puedo identificar de forma única a la tupla de la relación

Llaves

Superllave

- El conjunto de TODOS los atributos de la relación siempre forma una super llave

Productor	Cepa	Gama	Año	Grados	Orígen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Tarapaca	Merlot	Reserva	2011	14	San Pedro
Viu Manent	Carmenere	Gran Reserva	2014	12	Colchagua
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	14	Maipo
...	...	...	...	...	...

- Otras super llaves:
 - (Productor, Cepa, Gama, Año, Grados)
 - (Productor, Cepa, Gama, Año, Orígen)

Llaves

Superllave

- Como (Productor, Cepa, Gama, Año, Orígen) es superllave, lo siguiente no está permitido en una instancia:

Productor	Cepa	Gama	Año	Grados	Orígen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Tarapaca	Merlot	Reserva	2011	14	San Pedro
Viu Manent	Carmenere	Gran Reserva	2014	12	Colchagua
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	14	Maipo
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	13	Maipo

Repite (Productor, Cepa, Gama, Año, Orígen) con distinto Grados

Llaves

Llave candidata

- La llave (o llave candidata) para una relación R es:
 - *un conjunto de atributos de R que es una super llave de R , y para el cual no existe un subconjunto propio que también sea una super llave*
- Intuitivamente: una super llave que no se puede achicar

Llaves

Llave

Productor	Cepa	Gama	Año	Grados	Origen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Tarapaca	Merlot	Reserva	2011	14	San Pedro
Viu Manent	Carmenere	Gran Reserva	2014	12	Colchagua
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	14	Maipo
...	...	...	...	...	...

(Productor, Cepa, Gama, Año)

Llaves

Llave primaria

- La llave primaria es una llave candidata que queremos destacar, y la subrayamos en el esquema

Llaves

Llave primaria

Productor	Cepa	Gama	Año	Grados	Origen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Tarapaca	Merlot	Reserva	2011	14	San Pedro
Viu Manent	Carmenere	Gran Reserva	2014	12	Colchagua
Perez-Cruz	Cabernet Sauvignon	Edición Especial	2014	14	Maipo
...	...	...	...	...	...

Vinos(Productor, Cepa, Gama, Año, Grados, Origen)

Llaves

Si defino mal la llave primaria, habrá problemas

Productor	Cepa	Gama	Año	Grados	Origen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2015	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2014	14	Maipo
...	...	...	...	...	...

Vinos(Productor, Cepa, Gama, Año, Grados, Origen)

Un productor no puede producir vino de la misma cepa y gama en distintos años

Llaves

Si defino mal la llave primaria, tendré problemas

Productor	Cepa	Gama	Año	Grados	Origen
Perez-Cruz	Cabernet Sauvignon	Reserva	2014	13.5	Maipo
Perez-Cruz	Carmenere	Reserva	2014	13.5	Maipo
Perez-Cruz	Cabernet Sauvignon	Gran Reserva	2014	14	Maipo
...	...	...	...	...	...

Vinos(Productor, Cepa, Gama, Año, Grados, Origen)

Un productor no puede producir vinos de distinta cepa o gama en un mismo año

Llaves

Terminología

- **Super llave (superkey):** cualquier conjunto de atributos que determina a todo el resto
- **Llave (candidata):** cualquier conjunto de atributos que determina a todo el resto, y ninguno de sus subconjuntos propios es una super llave
- **Llave primaria:** una llave candidata que queremos destacar (la subrayada en el esquema)

Modelo Relacional

Llaves

- **Mejor tipo de llave: un ID único de la tupla**
- En la tabla Vinos no existía uno
- **Surrogate key (llave sustituta):** una llave genérica que simplifica las cosas → id
- Hay que tener cuidado con la lógica de la relación

Vinos(id, Productor, Cepa, Gama, Año, Grados, Orígen)

Llaves

Persona(id, rut, nombre)

Persona(id, rut, nombre)

- **Llave primaria: id**
- Otros ejemplos comunes de surrogate key:
 - Número de cliente
 - Número de servicio
 - SKU

Llaves

Ejemplo

Persona(id, rut, nombre)

- **Llave primaria:** id
- Llaves candidatas:
 - id
 - rut

Llaves

Personas

Persona(id, rut, nombre)

- **Superllaves:**
 - id
 - rut
 - id, rut
 - id, nombre
 - rut, nombre
 - id, rut, nombre
- **Llave primaria: id (surrogate key)**
- **Llaves candidatas:**
 - id
 - rut

Llaves

Tres esquemas relacionados

Cervezas(Nombre, Tipo, Grados, Ciudad-Origen)

Nombre	Tipo	Grados	Ciudad-Origen
Austral Lager	Lager	4.6	Punta Arenas
Austral Yagan	Ale	5.0	Punta Arenas
Kross 5	Ale	7.2	Curacaví

Vinos(Nombre, Tipo, Año, Grados, Ciudad-Origen)

Nombre	Tipo	Año	Grados	Ciudad-Origen
Tarapacá	Carmenere	2014	13.5	Maipo
Tarapacá	Merlot	2014	13.5	Maipo
Gato	Merlot	2016	14.0	Maule

En_Stock(Nombre, Cantidad, Precio_Unitario)

Nombre	Cantidad	Precio_Unitario

¿Cuál es la llave primaria más natural?

Llaves

Otro Ejemplo

Cervezas(Nombre, Tipo, Grados, Ciudad-Origen)

Nombre	Tipo	Grados	Ciudad-Origen
Austral Lager	Lager	4.6	Punta Arenas
Austral Yagan	Ale	5.0	Punta Arenas
Kross 5	Ale	7.2	Curacaví

Vinos(Nombre, Tipo, Año, Grados, Ciudad-Origen)

Nombre	Tipo	Año	Grados	Ciudad-Origen
Tarapacá	Carmenere	2014	13.5	Maipo
Tarapacá	Merlot	2014	13.5	Maipo
Gato	Merlot	2016	14.0	Maule

En_Stock(Nombre, Cantidad, Precio_Unitario)

Nombre	Cantidad	Precio_Unitario

Vinos y Cervezas pueden compartir Nombre — En-Stock no sabría a cuál producto se refiere

Llaves

Alternativa 1: nombre más específico para vinos

Nombre	Tipo	Grados	Ciudad-Origen
Austral Lager	Lager	4.6	Punta Arenas

Nombre	Tipo	Año	Grados	Ciudad-Origen
Tarapacá Carmenere 2014	Carmenere	2014	13.5	Maipo

Nombre	Cantidad	Precio-Unitario
Tarapacá Carmenere 2014	600	2000

Funciona, pero el nombre se vuelve largo y frágil como llave

Llaves

Alternativa 2: crear un id

Cervezas(id, Nombre, Tipo, Grados, Ciudad-Origen)

id	Nombre	Tipo	Grados	Ciudad-Origen
CauL00	Austral Lager	Lager	4.6	Punta Arenas

Vinos(id, Nombre, Tipo, Año, Grados, Ciudad-Origen)

id	Nombre	Tipo	Año	Grados	Ciudad-Origen
V TTC14	Tarapacá	Carmenere	2014	13.5	Maipo

En_Stock(id, Cantidad, Precio_Unitario)

id	Cantidad	Precio-Unitario
CauL00	600	2000
V TTC14	200	6000

id es una surrogate key generada — corta, estable y sin ambigüedad

Llaves

Alternativa 3: tablas En-Stock separadas

Cervezas(Nombre, Tipo, Grados, Ciudad-Origen)

Nombre	Tipo	Grados	Ciudad-Origen
Austral Lager	Lager	4.6	Punta Arenas
Austral Yagan	Ale	5.0	Punta Arenas
Kross 5	Ale	7.2	Curacaví

Vinos(Nombre, Tipo, Año, Grados, Ciudad-Origen)

Nombre	Tipo	Año	Grados	Ciudad-Origen
Tarapacá	Carmenere	2014	13.5	Maipo
Tarapacá	Merlot	2014	13.5	Maipo
Gato	Merlot	2016	14.0	Maule

Cerveza-En-Stock(Nombre, Cantidad, Precio-Unitario)

Nombre	Cantidad	Precio-Unitario
Austral Lager	600	2000

Vino-En-Stock(Nombre, Tipo, Año, Cantidad, Precio-Unitario)

Nombre	Tipo	Año	Cantidad	Precio-Unitario
Tarapaca	Merlot	2014	200	6500

Cada tabla de producto tiene su propia tabla de stock — sin ambigüedad, pero se duplica la estructura

Llaves

Alternativa 4: combinar las tablas

Cervezas(Nombre, Tipo, Grados, Ciudad-Origen, Cantidad, Precio-Unitario)

Nombre	Tipo	Grados	Ciudad-Origen	Cantidad	Precio-Unitario
Austral Lager	Lager	4.6	Punta Arenas	300	2000
Austral Yagan	Ale	5.0	Punta Arenas	600	?

Vinos(Nombre, Tipo, Año, Grados, Ciudad-Origen, Cantidad, Precio-Unitario)

Nombre	Tipo	Año	Grados	Ciudad-Origen	Cantidad	Precio-Unitario
Tarapaca	Carmenere	2014	4.6	Punta Arenas	200	6000
Tarapaca	Merlot	2014	5.0	Punta Arenas	?	6500
Gato	Merlot	2016	14.0	Maule	150	?

Se pierde la separación entre catálogo y stock — cada fila mezcla ambos conceptos

Del Diagrama E/R al Modelo Relacional

Diseño de base de datos

Análisis de requisitos

Usuarios



Requisitos

Diseño conceptual de bases de datos

Requisitos



Modelo entidad-relación

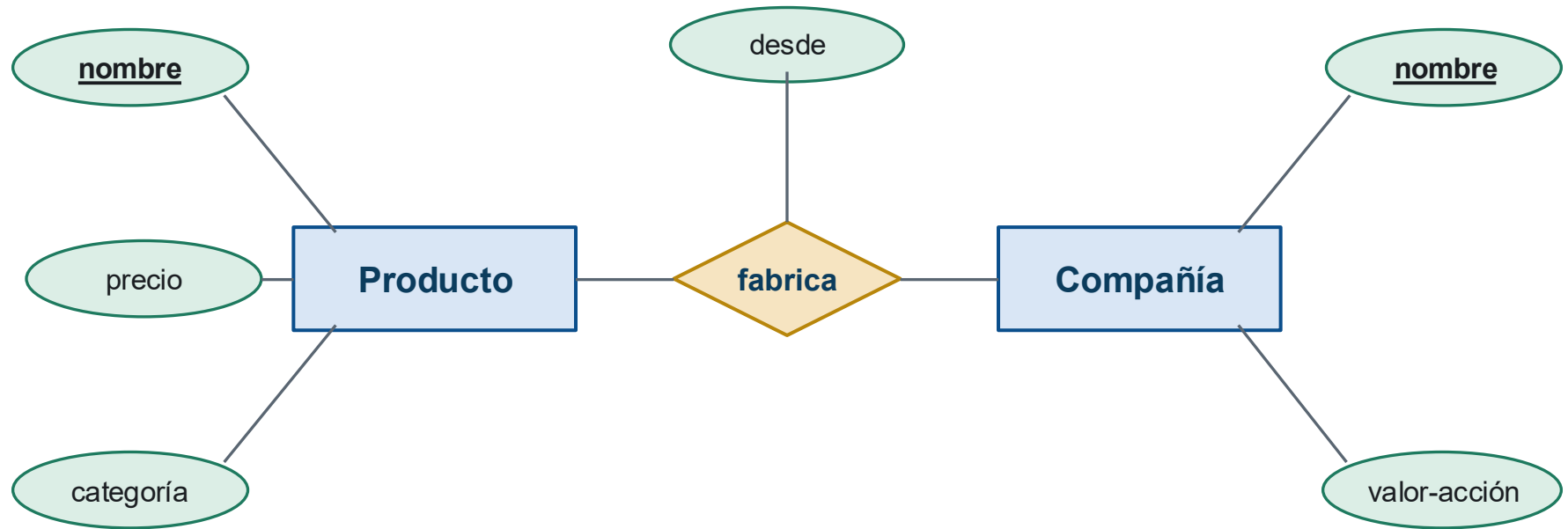
Diseño lógico de bases de datos

Modelo entidad-relación



Modelo relacional

M. E/R → M. Relacional



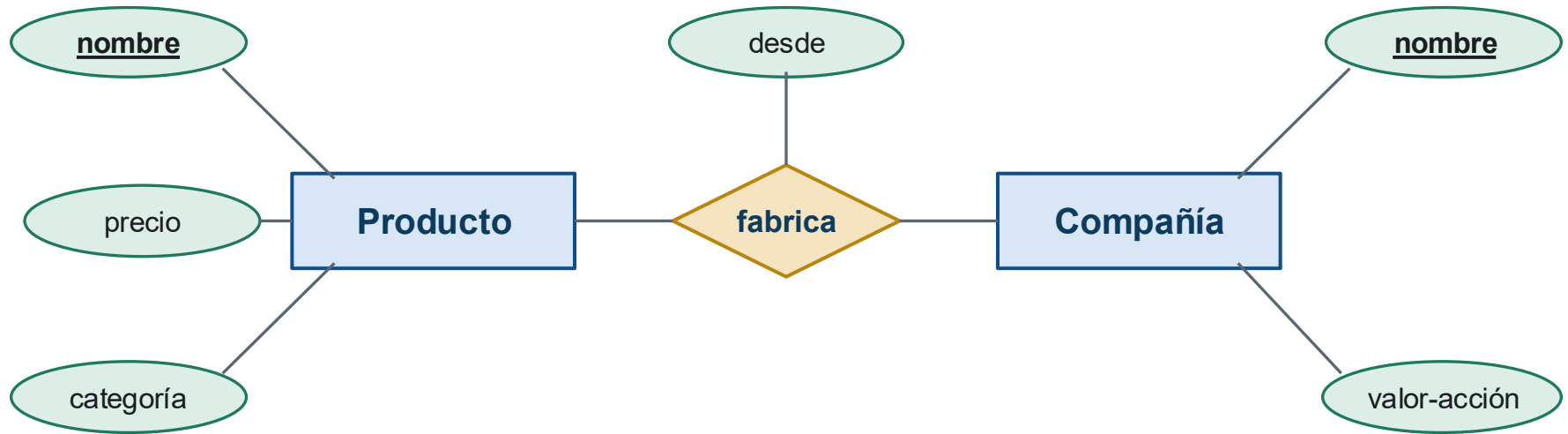
Las llaves de las entidades involucradas forman una super llave para la relación

```
Producto(nombre: string, precio: int, categoría: string)
```

```
Compañía(nombre: string, valor-acción: int)
```

```
Fabrica(Producto.nombre: string, Compañía.nombre, desde: date)
```

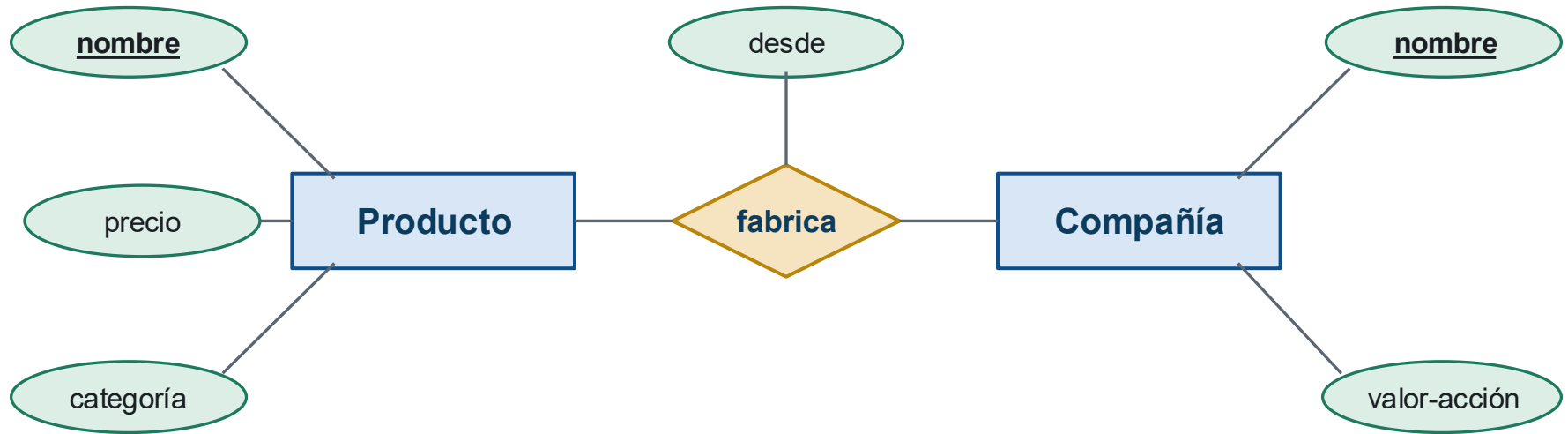

M. E/R → M. Relacional



Producto(nombre: string, precio: int, categoría: string)

```
CREATE TABLE producto(  
    nombre varchar(30),  
    precio int,  
    categoria varchar(30),  
    PRIMARY KEY (nombre)  
)
```

M. E/R → M. Relacional

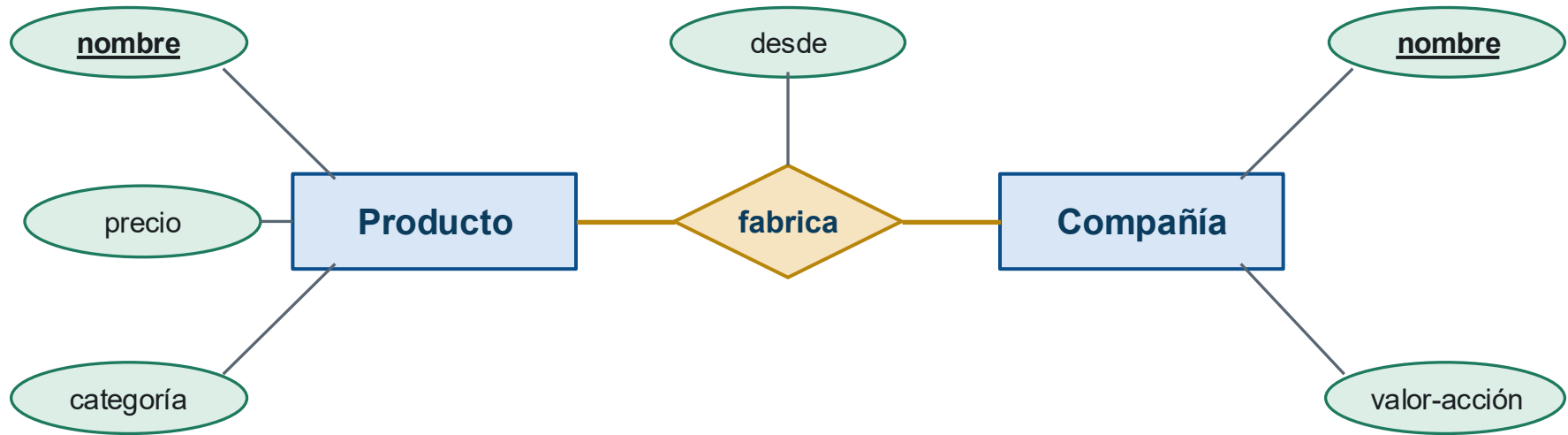


Compañía(nombre: string, valor-acción: int)

```
CREATE TABLE compania(  
    nombre varchar(30),  
    valor_accion int,  
    PRIMARY KEY (nombre)  
)
```

¿Cómo relacionamos la tabla compania con la tabla producto?

M. E/R → M. Relacional



Fabrica(Producto.nombre: string, Compañía.nombre, desde: date)

```
CREATE TABLE fabrica(  
    p_nombre varchar(30),  
    c_nombre varchar(30),  
    desde date,  
    PRIMARY KEY (p_nombre, c_nombre),  
    FOREIGN KEY(p_nombre) REFERENCES producto(nombre),  
    FOREIGN KEY(c_nombre) REFERENCES compania(nombre)  
)
```

**¡Usando llaves
foráneas!**

Paréntesis: Llaves foráneas

Llaves Foráneas

¿Qué es una llave foránea?

- Cuando la referencia a la tabla es una llave:

$$R[A_1, \dots, A_n] \subseteq S[B_1, \dots, B_n]$$

- Y $B_1, \dots, B_n$ son llave para S
- La relación R contiene la llave de la relación S
- Pero hay que tener cuidado, por ejemplo:

Llaves foráneas

¿Qué pasa aquí?



Producto

pid	nombre	precio
1	SonyXZ89	\$100.000
2	SonyXperia1	\$1.000.000
3	Huawei23	\$1000

Fabrica

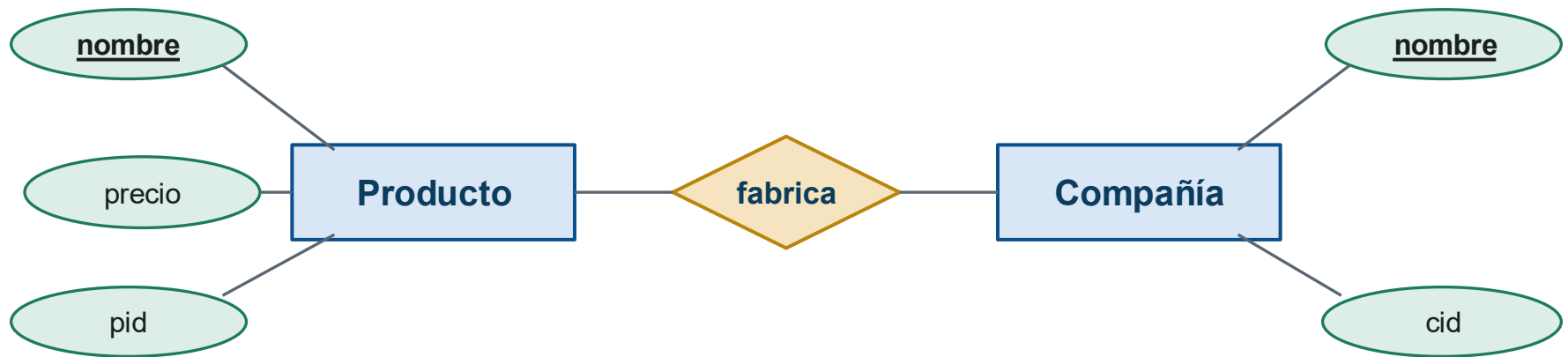
pid	cid
1	C1
2	C1
3	C1
89	C8

Compañía

cid	nombre
C1	Sony
C2	Apple

Llaves foráneas

¿Qué pasa aquí?



Producto

pid	nombre	precio
1	SonyXZ89	\$100.000
2	SonyXperia1	\$1.000.000
3	Huawei23	\$1000

Fabrica

pid	cid
1	C1
2	C1
3	C2
89	C8

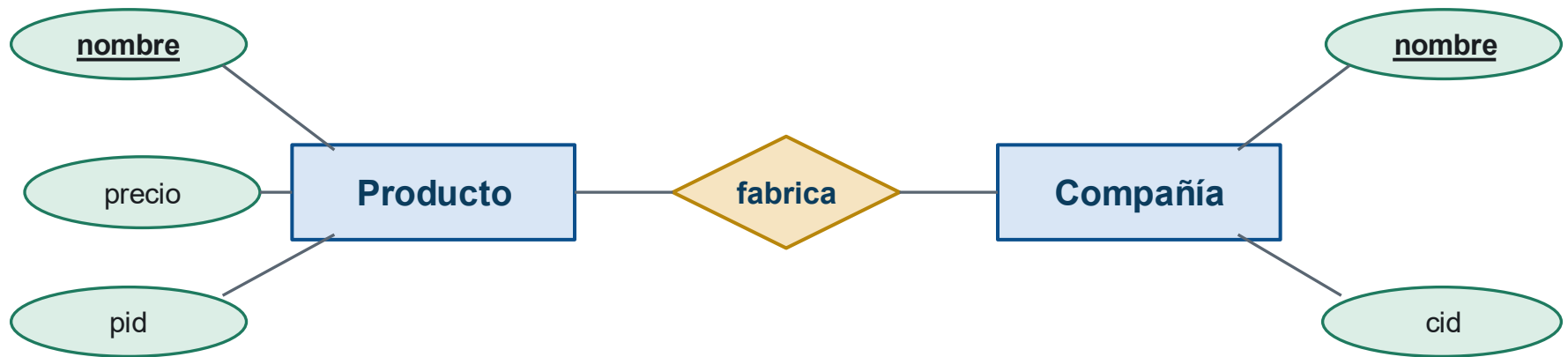
Compañía

cid	nombre
C1	Sony
C2	Apple

C8 no existe en la tabla compania

Llaves foráneas

¿Qué pasa aquí?



Producto

pid	nombre	precio
1	SonyXZ89	\$100.000
2	SonyXperia1	\$1.000.000
3	Huawei23	\$1000

Fabrica

pid	cid
1	C1
2	C1
3	C2
89	C8

Compañía

cid	nombre
C1	Sony
C2	Apple

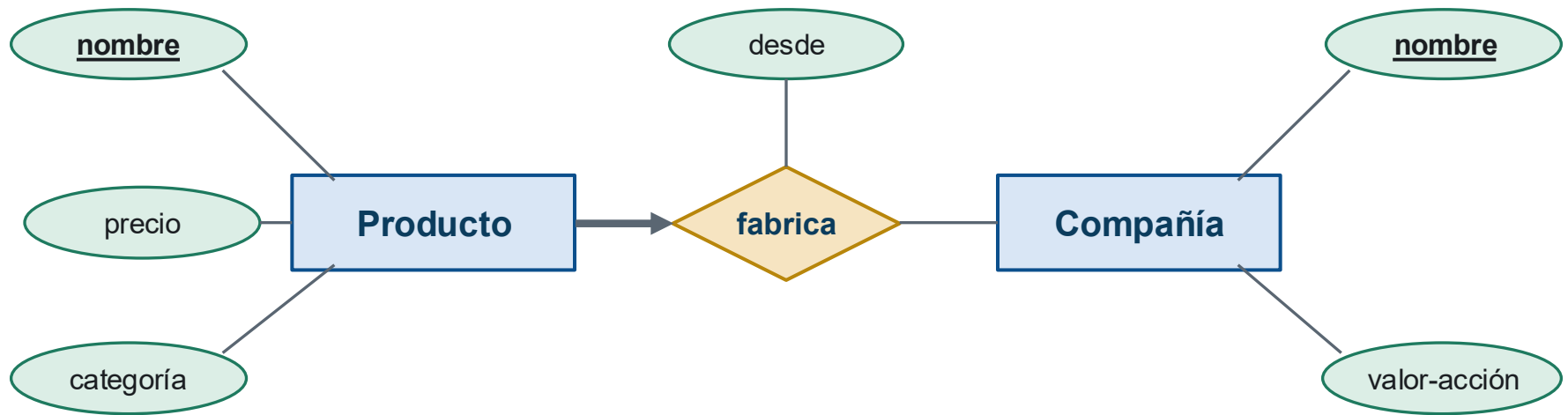
89 no existe en la tabla producto

C8 no existe en la tabla compania

**¿Cómo representar E/R con
llaves foráneas?**

M. E/R → M. Relacional

¿Qué hacemos en este caso?



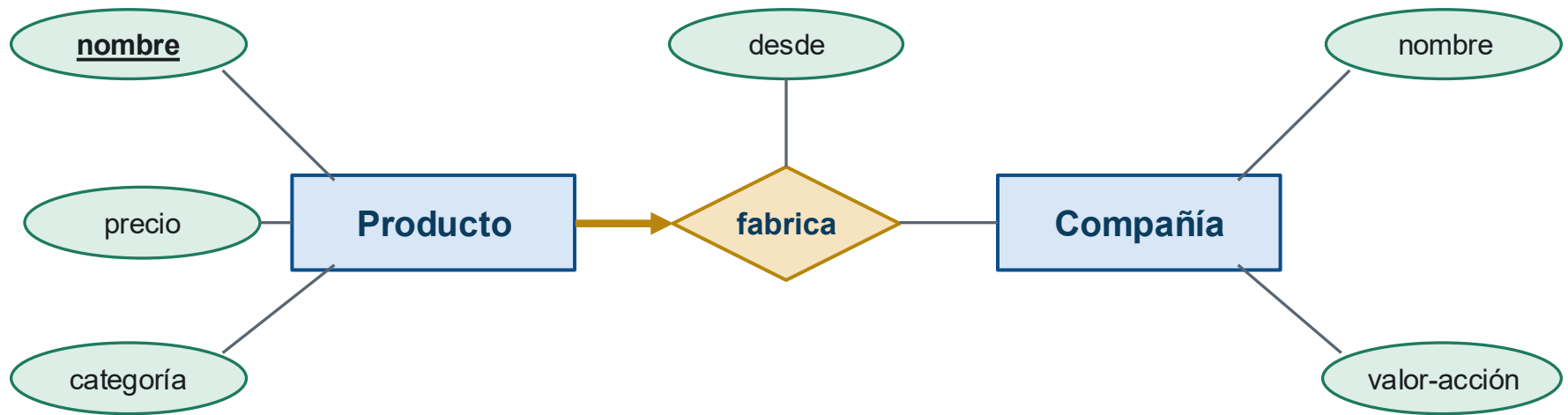
```
Producto(nombre: string, precio: int, categoría: string)
```

```
Compañía(nombre: string, valor-acción: int)
```

```
Fabrica(Producto.nombre: string, Compañía.nombre, desde: date)
```

M. E/R → M. Relacional

¿Qué hacemos en este caso?



```
Producto(nombre: string, precio: int, categoría: string)
```

```
Compañía(nombre: string, valor-acción: int)
```

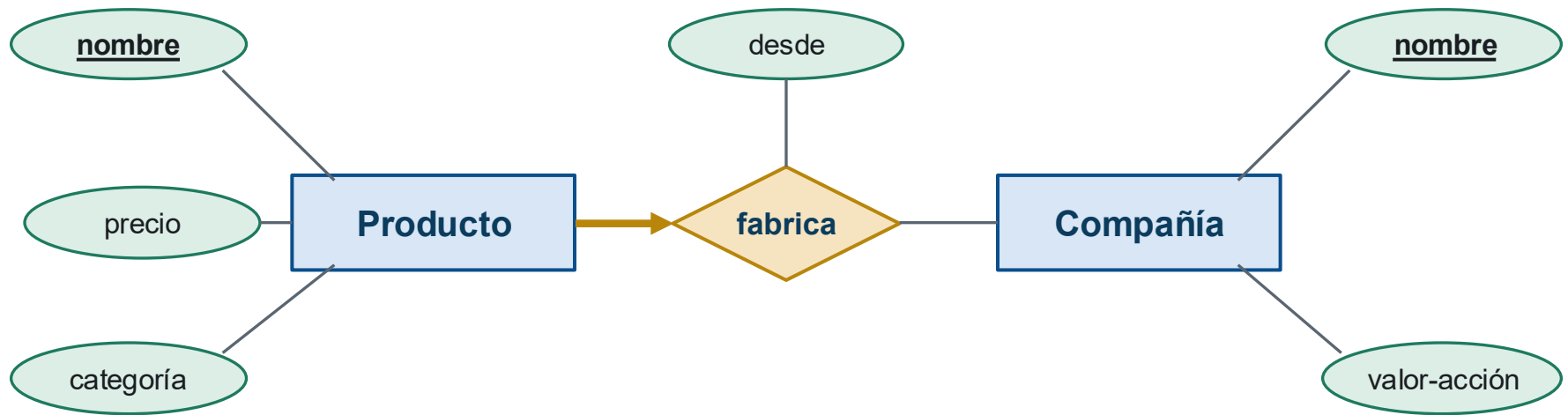
```
Fabrica(Producto.nombre: string, Compañía.nombre, desde: date)
```

Producto.nombre forma una llave candidata

No se necesita que Compañía.nombre sea llave

M. E/R → M. Relacional

¿Y ahora?



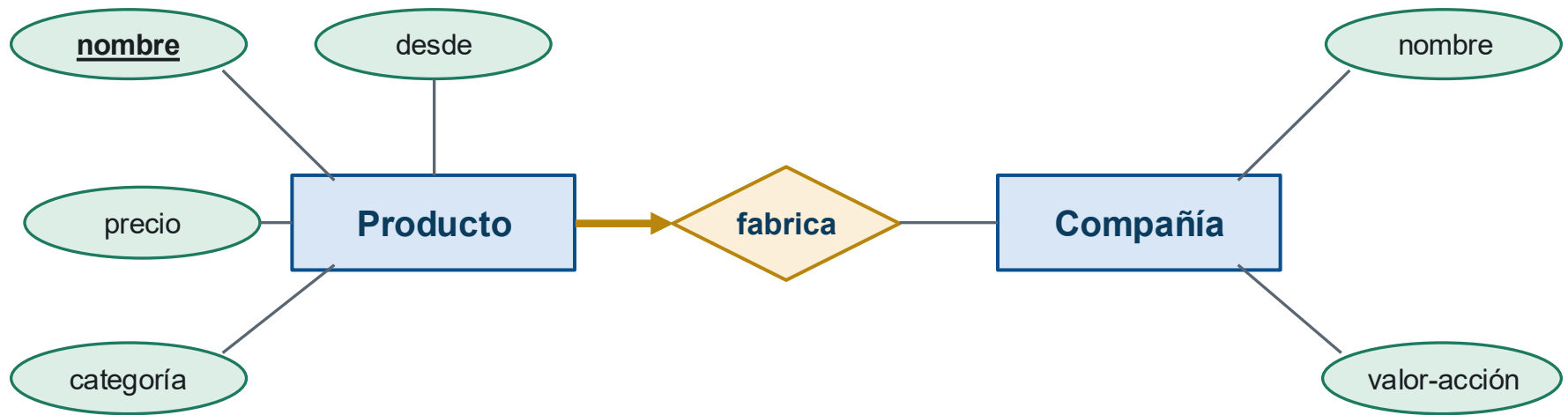
`Producto(nombre: string, precio: int, categoría: string, Compañía.nombre: string, desde: date)`

`Compañía(nombre: string, valor-acción: int)`

Sólo necesitamos una llave foránea en Producto.
Agregamos también el atributo de la relación.

M. E/R → M. Relacional

Un mejor diagrama...



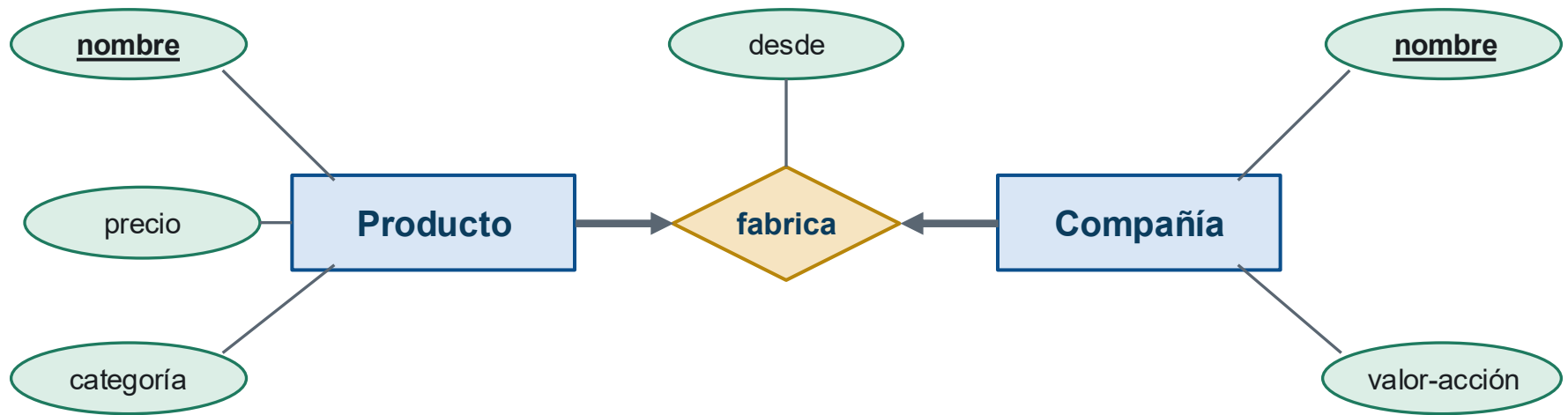
`Producto(nombre: string, precio: int, categoría: string, Compañía.nombre: string, desde: date)`

`Compañía(nombre: string, valor-acción: int)`

Sólo necesitamos una llave foránea en Producto.
Agregamos también el atributo de la relación.

M. E/R → M. Relacional

¿Y en este caso?



Producto(nombre: string, precio: int, categoría: string)

Compañía(nombre: string, valor-acción: int)

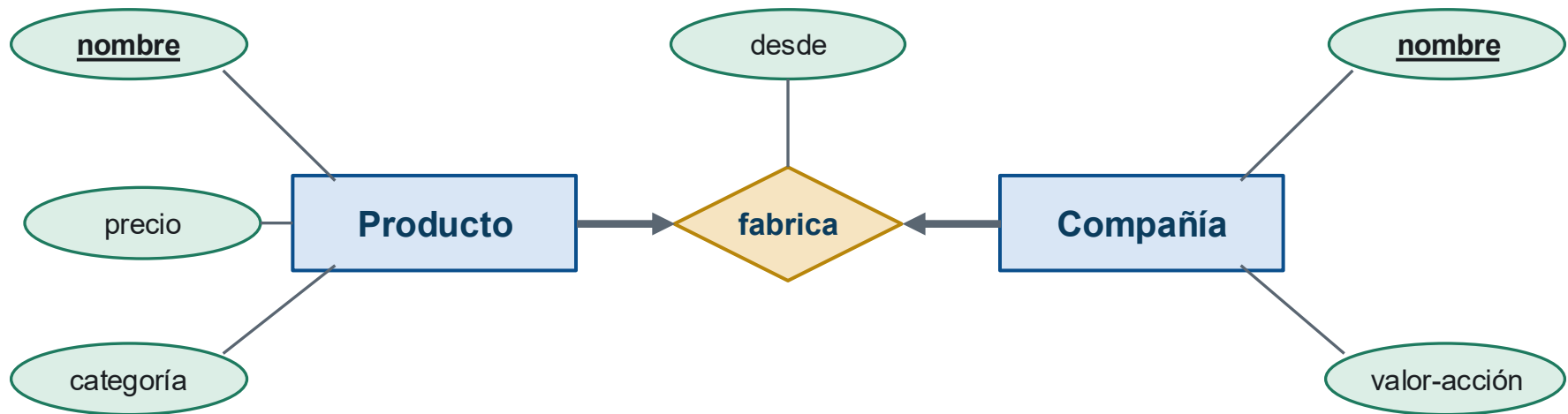
Fabrica(Producto.nombre: string, Compañía.nombre, desde: date)

X

Podemos hacer llave a Producto.nombre o a Compañía.nombre y agregar restricción con SQL UNIQUE

M. E/R → M. Relacional

¿Y en este caso?



`Producto(nombre: string, precio: int, categoría: string, Compañía.nombre: string, desde: date)`

`Compañía(nombre: string, valor-acción: int)`

“0”

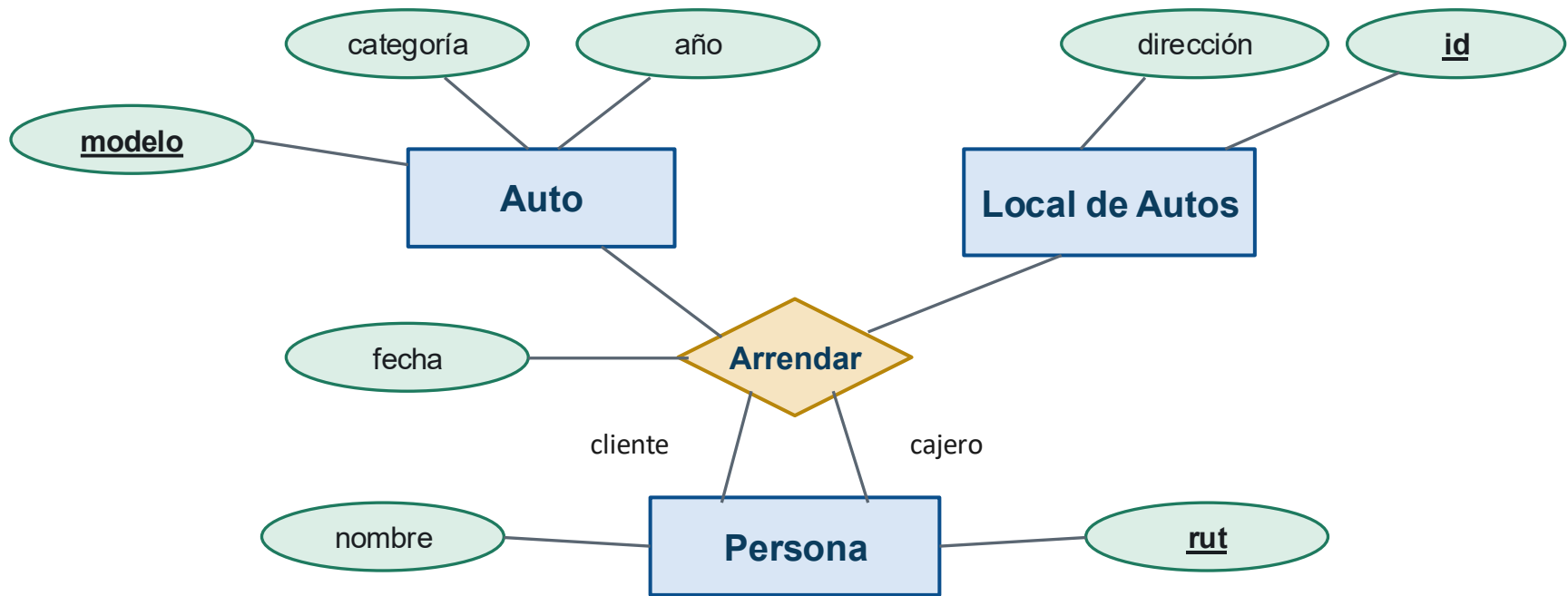
`Producto(nombre: string, precio: int, categoría: string)`

`Compañía(nombre: string, valor-acción: int, Producto.nombre: string, desde: date)`

Podemos hacer llave a `Producto.nombre` o a `Compañía.nombre` y agregar restricción con SQL UNIQUE

M. E/R → M. Relacional

¿Qué hacemos en este caso?



```
Auto(modelo: string, año: int, categoría: string)
```

```
LocalDeAutos(id: int, dirección: string)
```

```
Persona(rut: string, nombre: string)
```

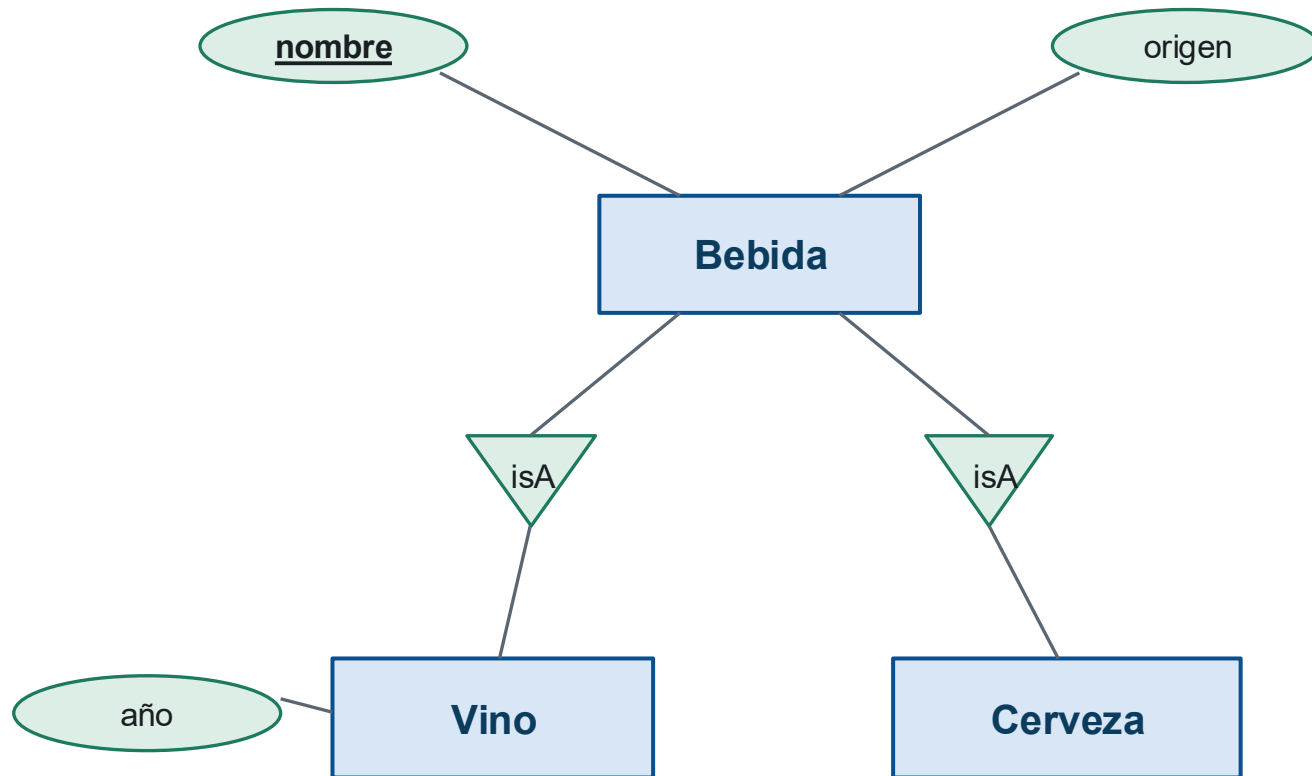
```
Arrendar(A.modelo: string, A.año: int, Pr.rut-cliente: string, Pr.rut-cajero: string, L.id: int, fecha: date)
```


**¿Cómo representar un M. E/R
en un**

M. Relacional con jerarquía de clases?

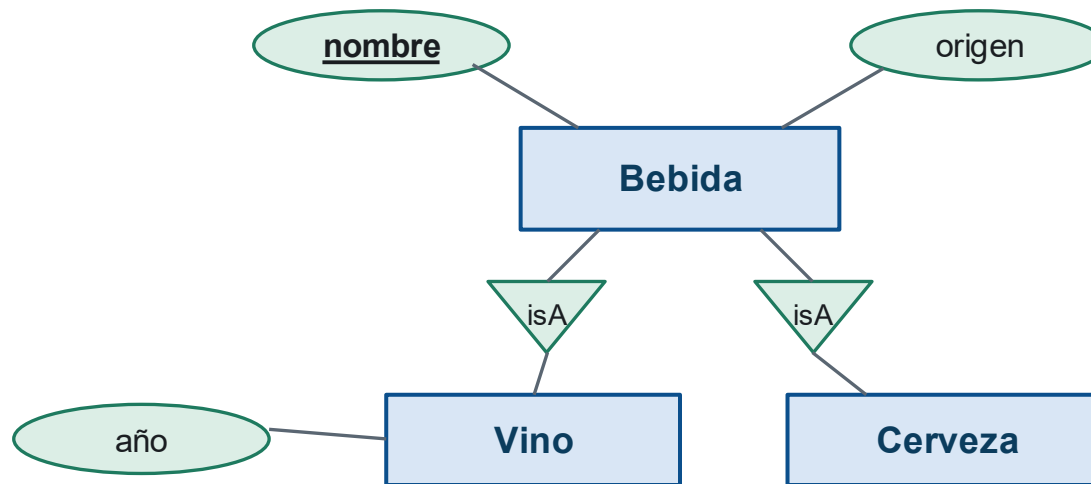
Ejemplo

Jerarquía de clases



M. E/R → M. Relacional

Jerarquía de clases



- Opción 1: tablas sólo para las subclases**

`Vino(nombre: string, origen: string, año: string)` • `Cerveza(nombre: string, origen: string)`

- Opción 2: tabla para la superclase**

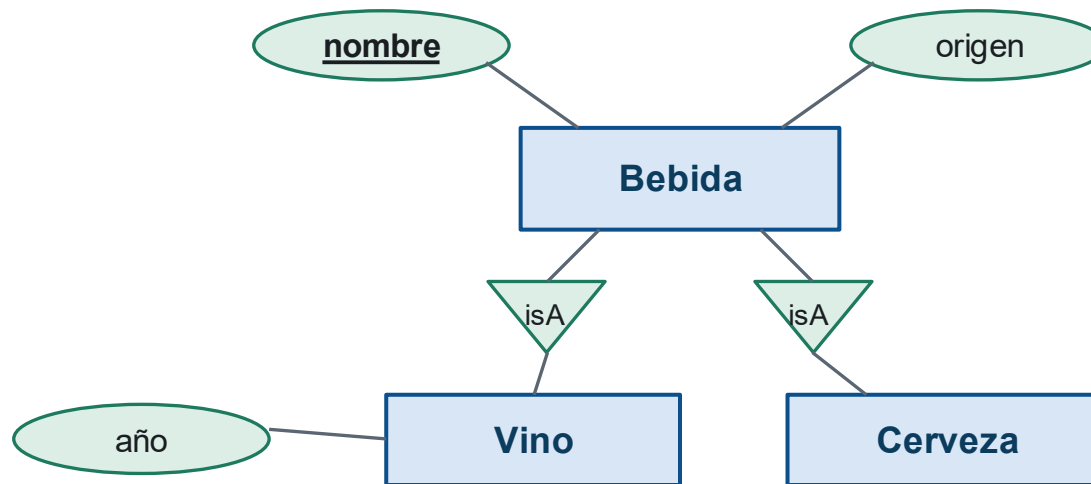
`Bebida(nombre: string, origen: string)` • `Vino(nombre: string, año: string)` •
`Cerveza(nombre: string)`

Se requieren joins para acceder a todos los datos

¿Cuál es mejor? Si hay mucho solapamiento → opción 2 (si no, hay mucha repetición de datos)

M. E/R → M. Relacional

Jerarquía de clases



- Opción 1: tablas sólo para las subclases**

`Vino(nombre: string, origen: string, año: string)` • `Cerveza(nombre: string, origen: string)`

- Opción 2: tabla para la superclase**

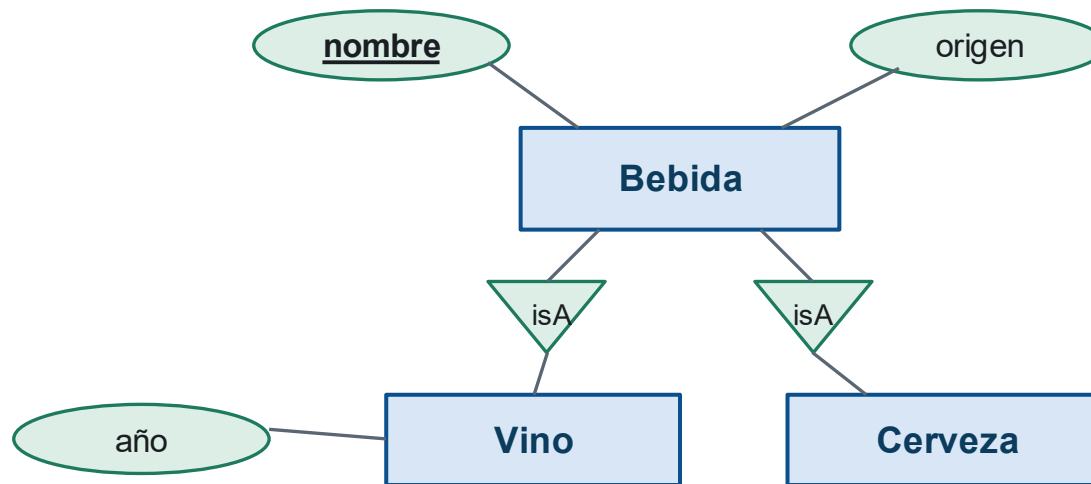
`Bebida(nombre: string, origen: string)` • `Vino(nombre: string, año: string)` •
`Cerveza(nombre: string)`

Se requieren joins para acceder a todos los datos

¿Cuál es mejor? Si no hay cobertura → opción 2 (o no podríamos guardar el whisky)

M. E/R → M. Relacional

Jerarquía de clases



- Opción 1: tablas sólo para las subclases**

`Vino(nombre: string, origen: string, año: string)` • `Cerveza(nombre: string, origen: string)`

- Opción 2: tabla para la superclase**

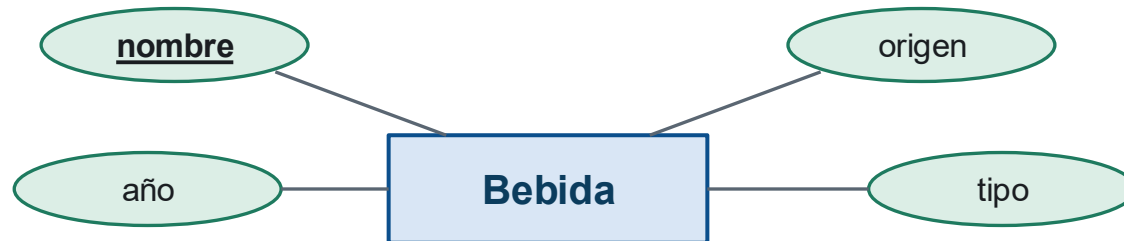
`Bebida(nombre: string, origen: string)` • `Vino(nombre: string, año: string)` •
`Cerveza(nombre: string)`

Se requieren joins para acceder a todos los datos

¿Cuál es mejor? Si hay muchas consultas por nombre → opción 2 (con la 1 habría que consultar dos tablas)

M. E/R → M. Relacional

Jerarquía de clases



- **Opción 3: quitar la jerarquía**

`Bebida(nombre: string, origen: string, año: string, tipo: string)`

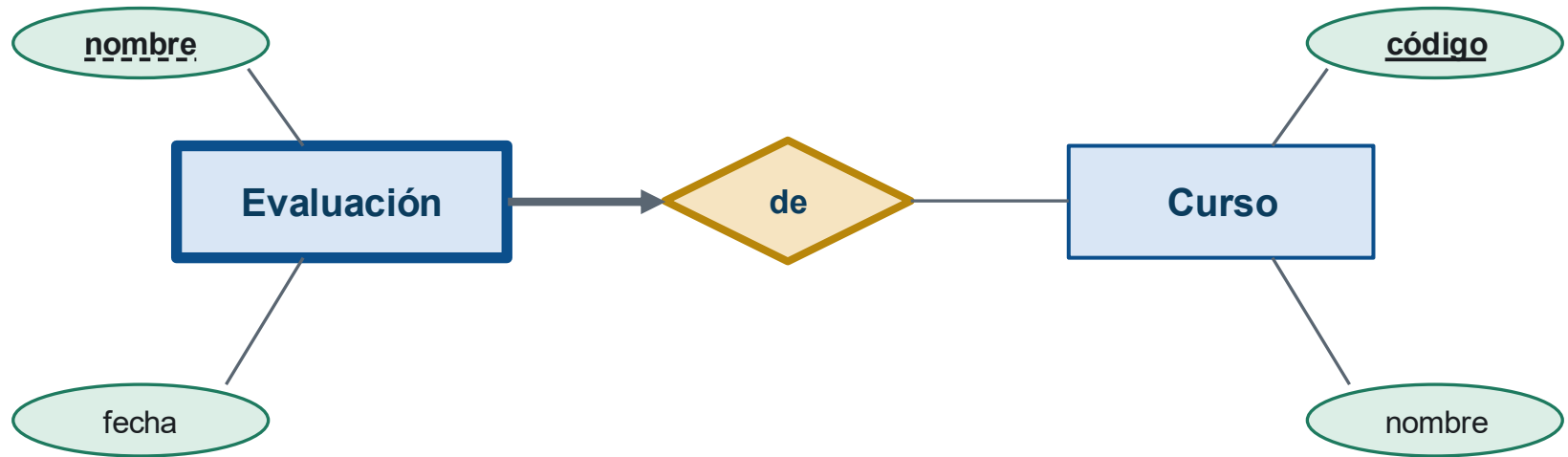
- Muchas repeticiones de la columna tipo
- Puede que no se conozca el tipo (nulls)
- **Pero es más sencillo (y comprimible)**

**¿Cómo representar un M. E/R
en un**

M. Relacional con entidades débiles?

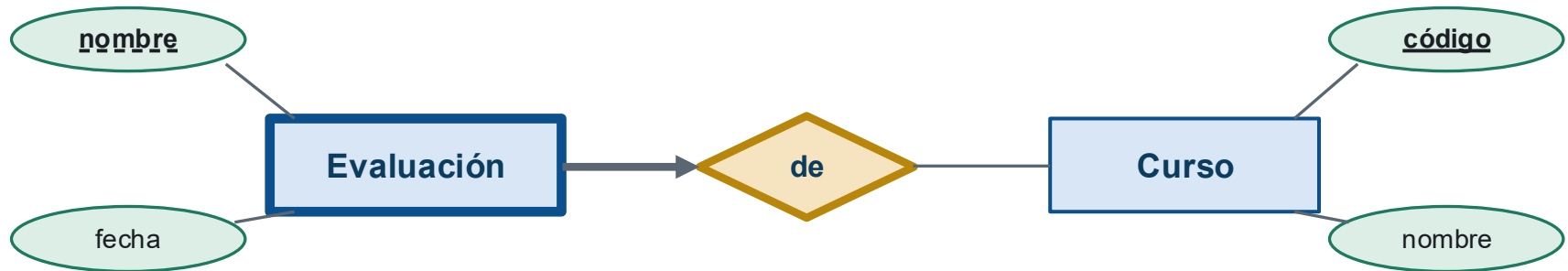
Ejemplo

Entidades débiles



M. E/R → M. Relacional

Entidades débiles



- ¿Qué tablas necesitamos?

Curso(codigo: string, nombre: string)

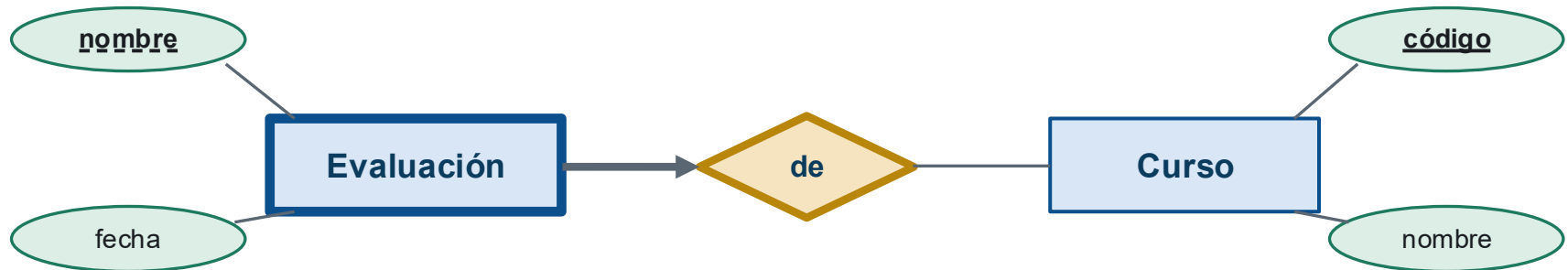
Evaluación(nombre: string, C.codigo: string, fecha: date)

De(E.nombre: string, C.codigo: string) ✗

¿Está bien esto? La tabla De es redundante (1-a-algo) y es mal nombre para una tabla

M. E/R → M. Relacional

Entidades débiles



Curso(codigo: string, nombre: string)

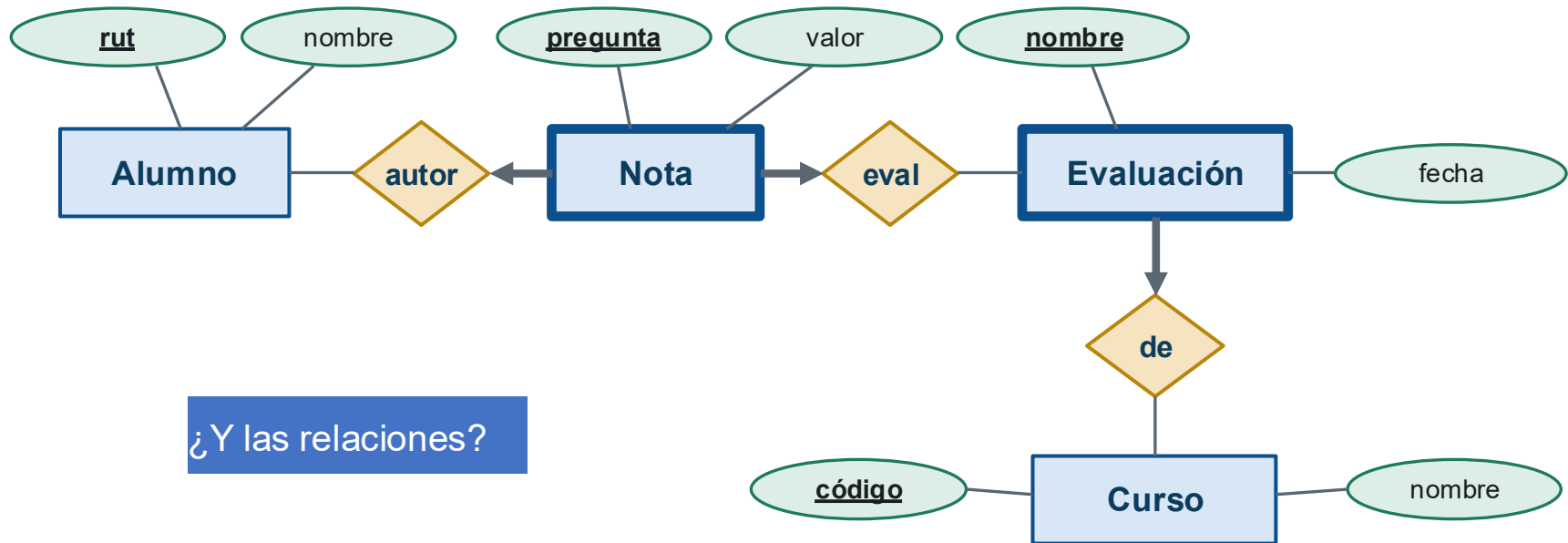
Evaluación(nombre: string, C.codigo: string, fecha: date)

```
CREATE TABLE evaluacion(  
    nombre varchar(30) NOT NULL,  
    codigo varchar(30) NOT NULL,  
    fecha date,  
    PRIMARY KEY (nombre, codigo)  
    FOREIGN KEY(codigo) REFERENCES curso(codigo) ON DELETE CASCADE  
)
```

¡Ahora Si!

M. E/R → M. Relacional

Entidades débiles



Curso(código: string, nombre: string)

Evaluación(nombre: string, C.código: string, fecha: date)

Nota(pregunta: int, E.nombre: string, C.código: string, A.rut: string, valor: float)

Alumno(rut: string, nombre: string)

**¿Cómo representar un M. E/R
en un**

M. Relacional con agregación?

Ejemplo: Arrendar un auto

- Un local de autos ofrece vehículos en arriendo
- Cada arriendo involucra a un cliente, un auto y un local, con fechas de inicio y término
- **¿Cómo modelamos esta relación de tres partes en el modelo relacional?**

Ejemplo: Arrendar un auto

Dirección

desde

hasta

disponibilidad

categoría

modelo

precio

Aeropuerto Arturo Merino Benítez
sáb, 30 mar 2024, 10:00

Aeropuerto Arturo Merino Benítez
mar, 2 abr 2024, 10:00

Modificar

53 coches disponibles

Ordenar por: Recomendado

Coche mediano Coche pequeño Coche grande SUV Monovolumen Familiar

Filtro [Borrar todos los filtros](#)

Lugar

- ☐ Aeropuerto (terminal) 44
- ☐ Aeropuerto (Meet & Greet) 4
- ☐ Aeropuerto (área de oficinas de alquiler de coches) 3
- ☐ Otras ubicaciones 2

Precio por día

- ☐ \$0 - \$50.000 8
- ☐ \$50.000 - \$100.000 39
- ☐ \$100.000 - \$150.000 6
- ☐ \$150.000 - \$200.000 0
- ☐ Más de \$200.000 0

Características del coche

- ☐ Aire acondicionado 51
- ☐ 4+ puertas 53

Coches eléctricos

- ☐ Totalmente eléctrico 0
- ☐ Híbrido 0
- ☐ Híbrido enchufable 0

Kilometraje

- ☐ Ilimitado 53

Transmisión

- ☐ Automática 37
- ☐ Manual 16

Volkswagen T-Cross o un SUV similar

5 plazas
1 maleta grande
Kilometraje ilimitado

Manual
1 maleta pequeña

Precio por 3 días:
\$124.294,00
Cancelación gratuita

Santiago Aeropuerto
En el aeropuerto

Ver oferta

Información importante

Enviar presupuesto por e-mail

MG One o un SUV similar

5 plazas
3 maletas grandes
Kilometraje ilimitado

Automática

Precio por 3 días:
\$143.551,00
Cancelación gratuita

Santiago Aeropuerto
En el aeropuerto

Ver oferta

Información importante

Enviar presupuesto por e-mail

Chery Tiggo o un SUV similar

5 plazas
3 maletas grandes
Kilometraje ilimitado

Manual

Precio por 3 días:
\$144.111,00
Cancelación gratuita

Santiago Aeropuerto
En el aeropuerto

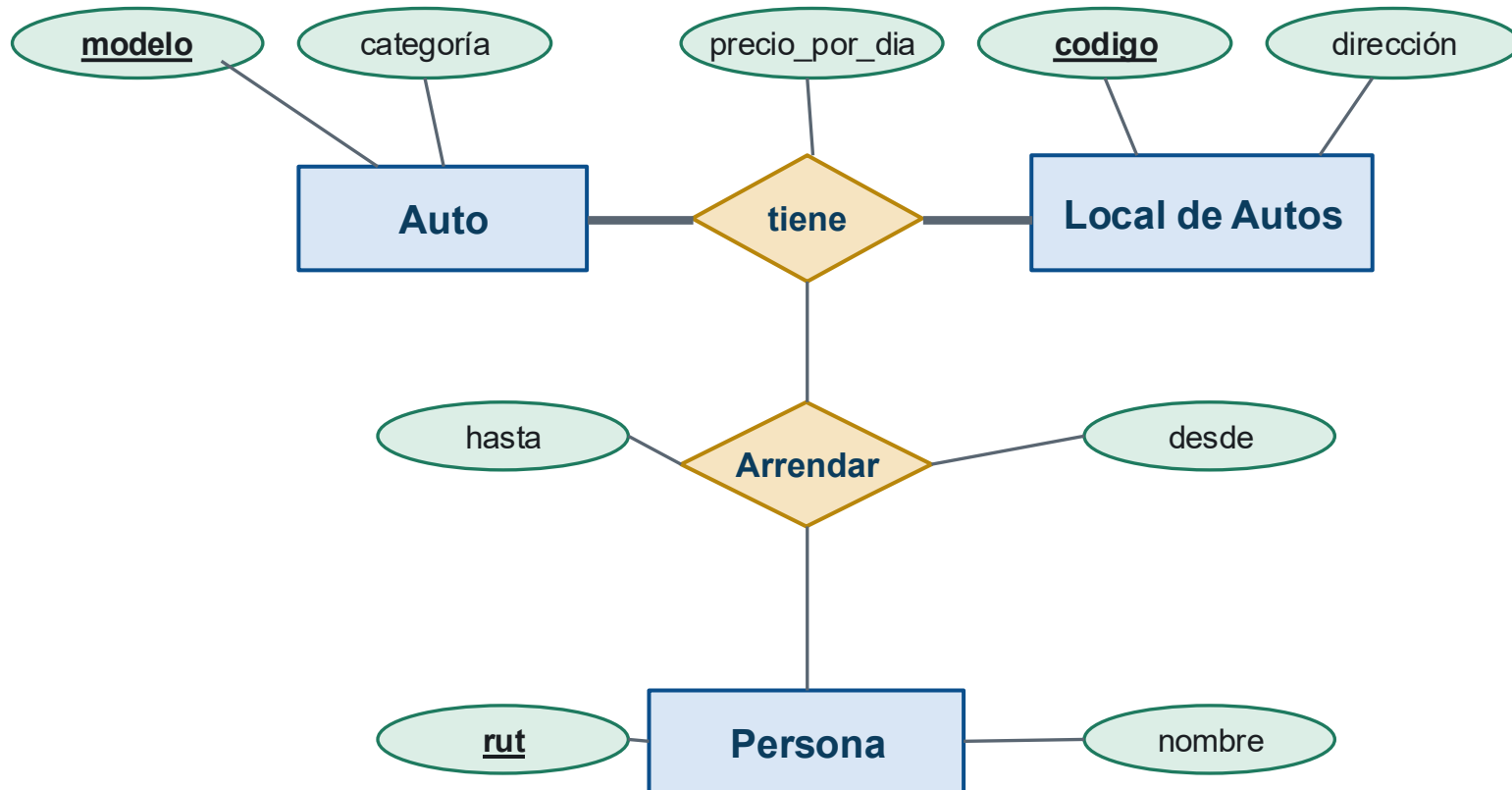
Ver oferta

Información importante

Enviar presupuesto por e-mail

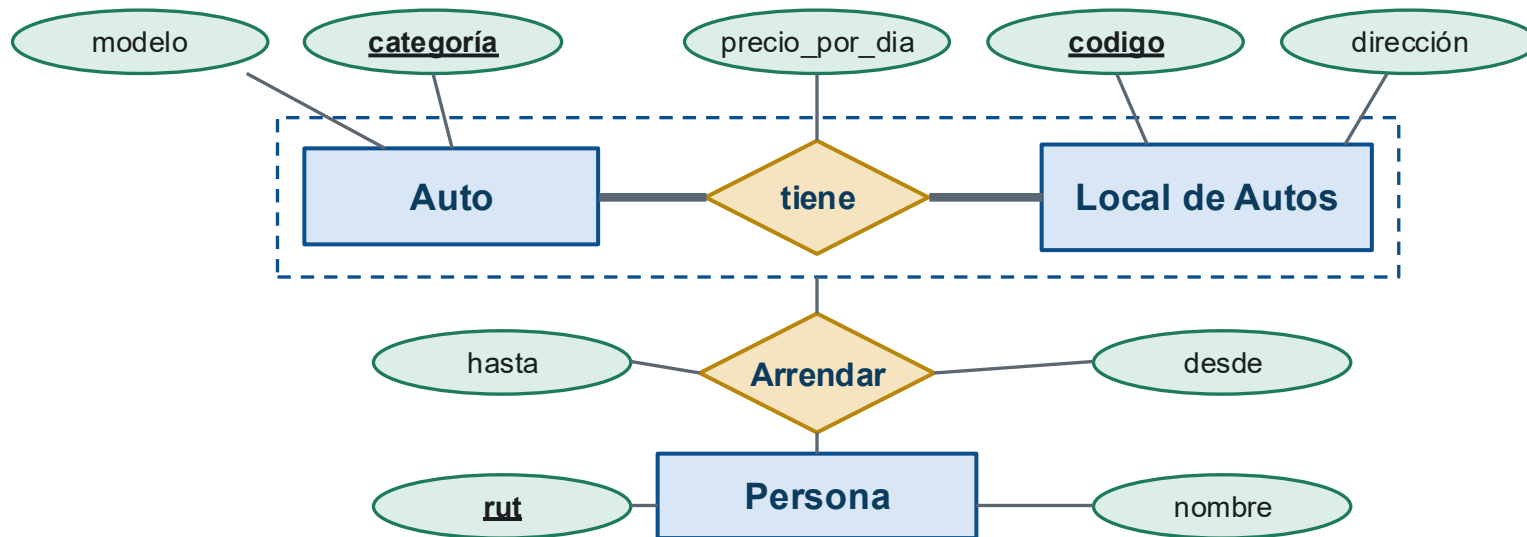
M. E/R → M. Relacional

Agregación



M. E/R → M. Relacional

Agregación



Auto(categoria: string, modelo: string) as A

LocalDeAutos(codigo: string, dirección: string) as L

Tiene(A.categoria, L.codigo: string, precio_por_dia: int) as T

Persona(rut: string, nombre: string) as P

Arrendar(T.a.categoria, T.l.codigo, P.rut, desde: date, hasta: date)

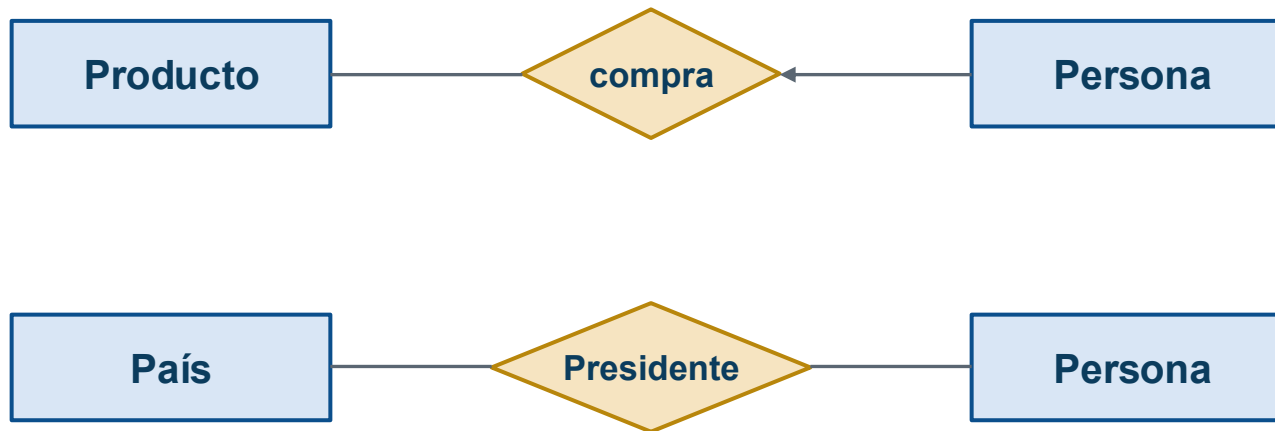
Falta:

- multiplicidades
- Entrega en dirección diferente

Principios básicos del diseño

Principios básicos del diseño

1. Fidelidad al problema

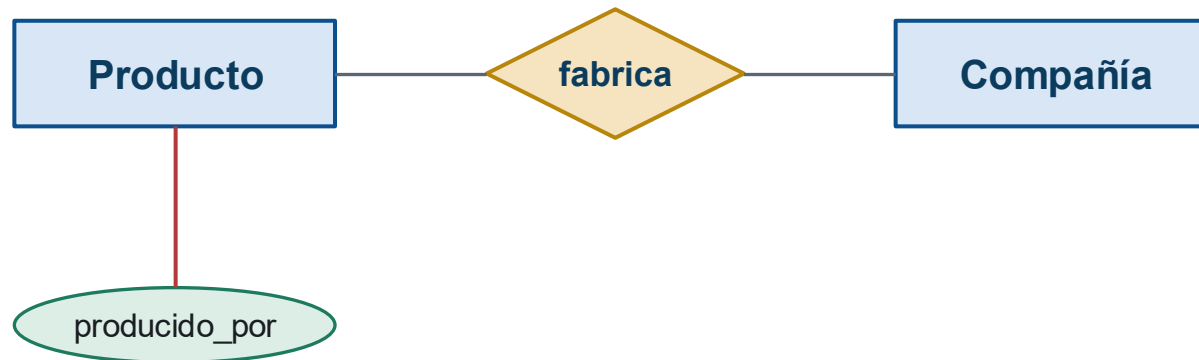


Las relaciones no tienen nada que ver entre sí

Principios básicos del diseño

2. Evitar redundancia

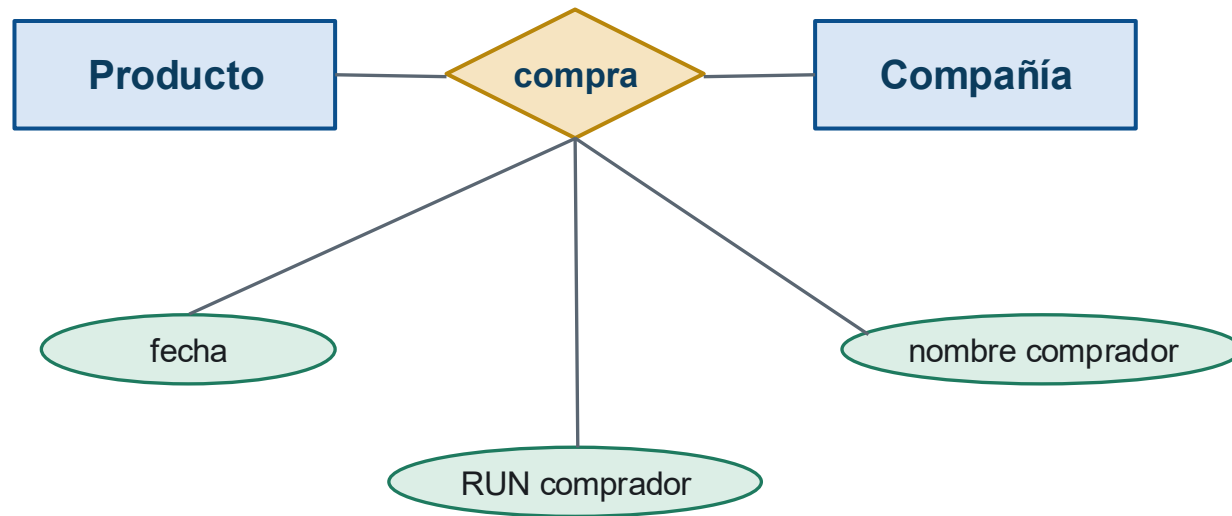
- ¿Qué está mal?



El atributo **producido_por** es redundante y puede generar anomalías

Principios básicos del diseño

3. Elegir entidades y relaciones correctamente

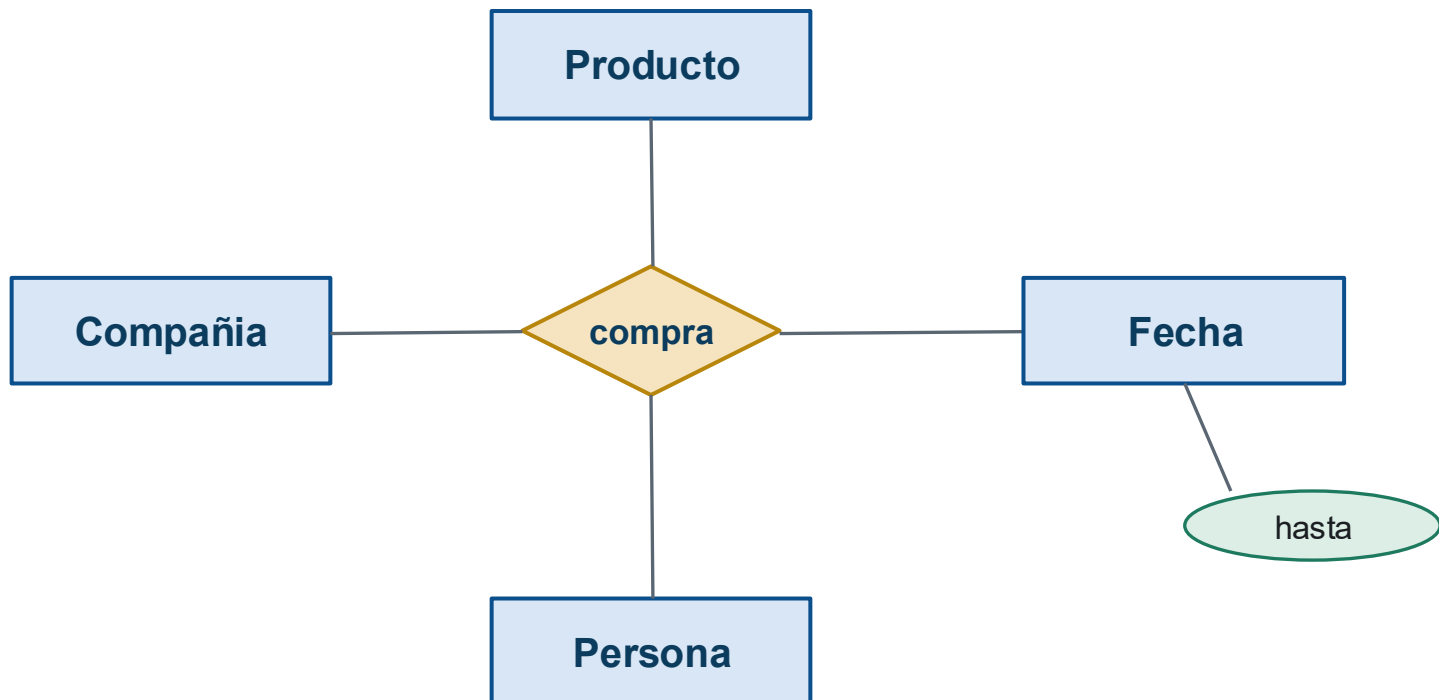


Sería mejor crear una entidad Persona que relacione el producto a través de la compra

Principios básicos del diseño

4. No complicar más de lo necesario

- ¿Qué está mal?



Fecha puede ser un atributo de compra, no una entidad aparte

Principios básicos del diseño

5. Buena elección de llave primaria

- Al diseñar, siempre queremos identificar los atributos de las entidades que son candidatos a ser llave: los llamamos natural key, porque naturalmente se comportan como una llave

`Usuario(email, rut, username, nombre, tipo, fecha_de_inscripcion)`

- *rut*, *email* y *username* son posibles natural keys
- ...pero en la práctica el 99% de las veces es mejor usar una columna inventada, sin significado, autogenerada por el RDBMS
- A esto le llamamos ***surrogate key***

Principios básicos del diseño

Elección de llave primaria

- Bueno, en realidad es algo medio opinionado...

Existe un debate extenso en la comunidad — "surrogate vs. natural/business keys" — sobre cuál conviene usar según el caso

Principios básicos del diseño

Elección de llave primaria

- Bueno, en realidad es algo medio opinionado...
- Pero en la práctica, los frameworks de desarrollo web modernos esperan una surrogate key llamada id como llave primaria, e incluso la generan por defecto
- La tabla anterior deberíamos generarla así:

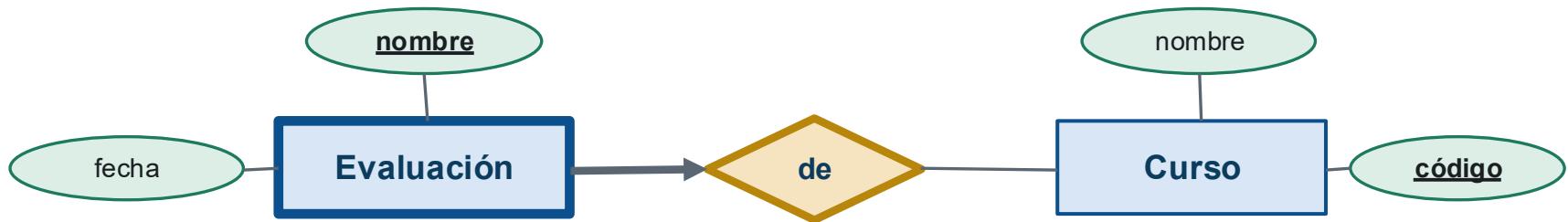
```
CREATE TABLE usuario(  
    id SERIAL,  
    email varchar(30) UNIQUE NOT NULL,  
    rut varchar(30) UNIQUE NOT NULL,  
    fecha date,  
    PRIMARY KEY (id)  
)
```


Restricciones de integridad

Restricciones de integridad

- Son restricciones formales que imponemos a un esquema que todas sus instancias deben satisfacer. Algunas son:
- **De valores nulos:** el valor puede o no ser nulo
- **Unicidad:** dado un atributo, no puede haber dos tuplas con el mismo valor
- **De llave:** el valor es único y no puede ser null
- **De referencia:** si se referencia una compañía, ésta debe existir (llaves foráneas)
- **De dominio:** la edad de las personas debe estar entre 0 y 150 años

M. E/R → M. Relacional



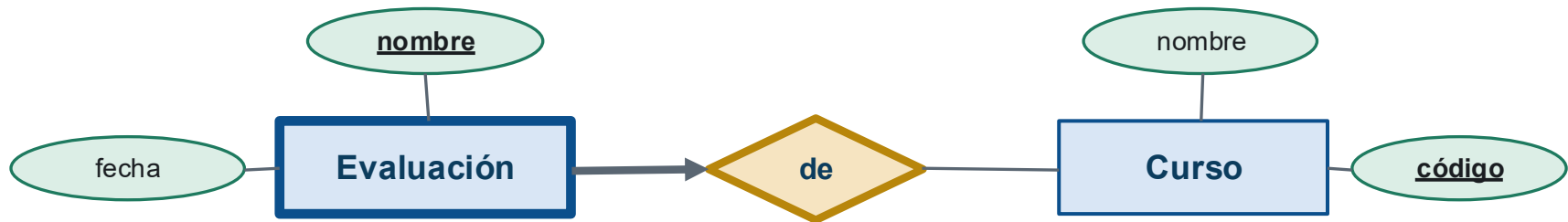
Curso(codigo: string, nombre: string)

Evaluación(nombre: string, C.código: string, fecha: date)

```
CREATE TABLE evaluacion(  
    nombre varchar(30) NOT NULL,  
    codigo varchar(30) NOT NULL,  
    fecha date DEFAULT NOW(),  
    PRIMARY KEY (nombre, codigo)  
    FOREIGN KEY(codigo) REFERENCES curso(codigo) ON DELETE CASCADE  
)
```

¿Qué restricciones ves aquí?

M. E/R → M. Relacional



Curso(codigo: string, nombre: string)

Evaluación(nombre: string, C.codigo: string, fecha: date)

```
CREATE TABLE evaluacion(  
    nombre varchar(30) NOT NULL,  
    codigo varchar(30) NOT NULL,  
    fecha date DEFAULT NOW(),  
    PRIMARY KEY (nombre, codigo)  
    FOREIGN KEY(codigo) REFERENCES curso(codigo) ON DELETE CASCADE  
)
```

- NOT NULL → no puede ser nulo · varchar(30) → a lo más 30 caracteres
- date → tiene que ser una fecha, con valor por defecto la fecha actual
- PRIMARY KEY (nombre, codigo) → la llave son nombre y código
- FOREIGN KEY(codigo) → codigo es llave foránea a la tabla curso
- ON DELETE CASCADE → borra las evaluaciones cuyo curso fue eliminado

Restricciones de integridad

Integridad de la entidad



- Un ejemplo de tabla de profesor con algunas restricciones:

```
CREATE TABLE Profesor(  
    id int PRIMARY KEY,  
    nombre varchar(30) NOT NULL,  
    apellidos varchar(30) NOT NULL,  
    telefono varchar(30) NOT NULL,  
    id_universidad int,  
    nivel varchar(20) DEFAULT 'Pregrado'  
    FOREIGN KEY(id_universidad) REFERENCES Universidad(id)  
);
```

Restricciones de integridad

Participación

- Cada profesor puede trabajar en una única universidad (pero puede estar sin trabajo):



- Cada profesor necesariamente trabaja en una única universidad:



Restricciones de integridad

Participación

- Cada profesor necesariamente trabaja en una única universidad



```
CREATE TABLE Profesor(  
    id int PRIMARY KEY,  
    nombre varchar(30) NOT NULL,  
    apellidos varchar(30) NOT NULL,  
    telefono varchar(30) NOT NULL,  
    id_universidad int NOT NULL,  
    nivel varchar(20) DEFAULT 'Pregrado'  
    FOREIGN KEY(id_universidad) REFERENCES Universidad(id)  
);
```

Restricciones de integridad

Dominio

- Queremos restringir el dominio de las columnas. Una forma simple de hacerlo en SQL es con CHECK:

```
CREATE TABLE Festival(  
    id int PRIMARY KEY,  
    nombre varchar(30) NOT NULL,  
    fecha_inicio date NOT NULL,  
    fecha_fin date NOT NULL,  
    precio int NOT NULL,  
    CHECK( precio BETWEEN 10000 AND 1000000 ),  
    CHECK( fecha_fin > fecha_inicio)  
)
```